

Java Foundations Exam Fundamentals

Exam Code 1Z0-811

Hanumant Deshmukh

Wednesday 10th September, 2025
Build 1.5

For online information and ordering of this book, please contact support@enthuware.com. For more information, please contact:

Hanumant Deshmukh
4A Agroha Nagar, A B Road,
Dewas, MP 455001
INDIA

Copyright © 2020 by Hanumant Deshmukh All Rights Reserved.

No part of this publication may be reproduced, stored in a retrieval system or transmitted in any form or by any means, electronic, mechanical, photocopying, recording, scanning or otherwise, except as permitted under the relevant laws of copyright, without the prior written permission of the Author. Requests for permission should be addressed to support@enthuware.com

Limit of Liability/Disclaimer of Warranty: The author makes no representations or warranties with respect to the accuracy or completeness of the contents of this work and specifically disclaim all warranties, including without limitation warranties of fitness for a particular purpose. No warranty may be created or extended by sales or promotional materials. The advice and strategies contained herein may not be suitable for every situation. This work is sold with the understanding that the author is not engaged in rendering legal, accounting, or other professional services. If professional assistance is required, the services of a competent professional person should be sought. The author shall not be liable for damages arising herefrom. The fact that an organization or website is referred to in this work as a citation and/or a potential source of further information does not mean that the author endorses the information the organization or Web site may provide or recommendations it may make. Further, readers should be aware that Internet Web sites listed in this work may have changed or disappeared between when this work was written and when it is read.

Lead Author and Publisher: Hanumant Deshmukh
Technical Editor: Liu Yang
Technical Validators: Anil Kumar, Bill Bruening
Technical Proofreaders: Carol Davis, Robert Nyquist
Copy Editor: Lisa Wright
Book Designers: Fatimah Arif
Proofreader: Ben Racca
Typesetter: Lillian Musambi
Cover Design: Kino A Lockhart, LOXarts Development, <http://www.loxarts.com>

TRADEMARKS: Oracle and Java are registered trademarks of Oracle America, Inc. All other trademarks are the property of their respective owners. The Author is not associated with any product or vendor mentioned in this book.

Wednesday 10th September, 2025 Build 1.5

To my alma mater,

Indian Institute of Technology, Varanasi

Acknowledgements

I would like to thank numerous individuals for their contribution to this book. Thank you to Liu Yang for being the Technical Editor and Lisa Wright for being the copy editor. Thank you to Carol Davis and Robert Nyquist for technical proof reading. Thank you to Aakash Jangid and Bill Bruening for validating all the code snippets in this book.

Thank you to Maaike Van Putten for her inputs on the book cover design and to Kino Lockhart for designing the cover.

This book also wouldn't be possible without many people at Enthware, including Paul A Prem, who have been developing mock exams for the past fifteen years. Their experience helped fine tune several topics in this book.

I would also like to thank Bruce Eckel, the author of "Thinking In Java" for teaching Java to me and countless others through his book.

I am also thankful to countless Enthware.com and CodeRanch.com forum participants for highlighting the topics that readers often find difficult to grasp and also for showing ways to make such topics easy to understand.

Thank you to Edward Dang, Rajashekar Kommu, Kaushik Das, Gopal Krishna Gavera, Dinesh Chinalachiagari, Jignesh Malavia, Michael Tsuji, Hok Yee Wong, Ketan Patel, Anil Malladi, Bob Silver, Jim Swiderski, Krishna Mangallampalli, Shishiraj Kollengreth, Michael Knapp, Rajesh Velamala, Aamer Adam, and Raghuveer Rawat for putting up with me :)

I would like to thank my family for their support throughout the time I was busy writing this book.

About the Author

Hanumant Deshmukh is a certified professional Java architect, author, and director of a software consultancy firm. Hanumant specializes in Java based multi-tier applications in financial domain. He has executed projects for some of the top financial companies. He started Enthuware.com more than twenty yrs ago through which he offers training courses and learning material for various Java certification exam. He has also co-authored a best selling book on Java Servlets/JSP certification, published by Manning in 2003.

Hanumant achieved his Bachelor of Technology from Institute of Technology, Banaras Hindu University (now, IIT - Varanasi) in Computer Science and his Masters in Financial Analysis from ICFAI. After spending more than a decade working with amazing people in the United States, he returned back to India to pursue a degree in Law. He is a big believer in freedom of speech and expression and works on promoting it in his spare time.

You may reach him at support@enthuware.com



Contents At A Glance

Introduction	i
1 Kickstarter for Beginners	1
2 What is Java?	7
3 Java Basics	19
4 Basic Java Elements	53
5 Working With Java Data Types	79
6 Working with Java Operators	105
7 Using Decision Statements	131
8 Using Looping Statements	149
9 Arrays and ArrayLists	173
10 Classes and Constructors	197
11 Java Methods	221
12 Working with the Math and Random Classes	245
13 Working with the String Class	251
14 Debugging and Exception Handling	269

Contents

Introduction	i
0.1 Who is this book for?	iii
0.2 How is this book different from others?	iii
0.3 How is this book organized?	v
0.4 General tips for the exam	vi
0.5 Official Exam Details and Exam Objectives	viii
0.6 Feedback, Errata, and Reviews	xi
1 Kickstarter for Beginners	1
1.1 Programming and Programming Languages	2
1.2 Applications	4
1.3 Application Programming Interfaces	5
2 What is Java?	7
2.1 Understanding Java	8
2.2 Features of Java	12
2.3 Real world Applications of Java	15
2.4 What Java is not!	16
3 Java Basics	19
3.1 Installing Java	20
3.1.1 System Check	20
3.1.2 Understanding Java Builds	23
3.1.3 Downloading and Installing OpenJDK 15	23
3.1.4 Using an Integrated Development Environment (IDE)	31
3.2 JDK and JRE	32
3.3 Components of a Java program	32
3.4 Compiling and running a basic Java program	34
3.5 Understanding the main method	36
3.5.1 The main method	36
3.5.2 Command line arguments	37
3.5.3 The end of main	38
3.6 Object-Oriented Programming	39
3.6.1 A matter of perspective	39
3.6.2 API	40
3.6.3 Principles of OOP	41

3.6.4	OOP in Java	43
3.6.5	Relation between Class, Object, and Reference	46
3.6.6	static and instance	49
3.7	Exercise	52
4	Basic Java Elements	53
4.1	Conventions	54
4.1.1	What is a Convention?	54
4.1.2	Conventions in Java	54
4.2	Java Identifiers and reserved words	55
4.2.1	Keywords, Literals, Reserved words, and Identifiers	55
4.2.2	Summary of important Java Keywords	56
4.3	Create and import packages	58
4.3.1	The package statement	58
4.3.2	Quiz	58
4.3.3	The import statement	59
4.3.4	Quiz	62
4.4	Structure of a Java class	63
4.4.1	Class disambiguated	63
4.4.2	Structure of a Java source file	63
4.4.3	Members of a class	63
4.4.4	Relationship between Java source file name and class name	66
4.4.5	Quiz	68
4.5	Advanced compilation and execution	69
4.5.1	Compilation and execution involving packages	69
4.5.2	Packaging classes into Jar	73
4.5.3	Compilation error vs exception at run time	74
4.6	The java.lang and other standard Java packages	75
4.7	Exercise	78
5	Working With Java Data Types	79
5.1	Data types in Java	80
5.1.1	Data types and Variable types	80
5.2	Difference between reference variables and primitive variables	84
5.2.1	Reference variables and primitive variables	84
5.3	Declare and initialize variables	86
5.3.1	Declaration and Definition	86
5.3.2	Declare and initialize variables	86
5.3.3	Uninitialized variables and Default values	88
5.3.4	Assigning values to variables	92
5.3.5	final variables	97
5.4	Declare and instantiate Java objects	98
5.4.1	Declare and instantiate Java objects	98
5.5	Use local variable type inference	101
5.5.1	Local variable type inference	101

5.6	Exercise	103
6	Working with Java Operators	105
6.1	Java Operators	106
6.1.1	Overview of operators available in Java	106
6.1.2	Expressions and Statements	116
6.1.3	Post and Pre Unary Increment/Decrement Operators	118
6.1.4	String concatenation using + and += operators	120
6.1.5	Numeric promotion and casting	122
6.1.6	Operator precedence and evaluation of expressions	125
6.2	Exercise	129
7	Using Decision Statements	131
7.1	Create if and if/else constructs	132
7.1.1	Basic syntax of if and if-else	132
7.1.2	Usage of if and if-else in the exam	133
7.2	Create ternary constructs	138
7.2.1	The ternary conditional operator ?:	138
7.3	Use a switch statement	141
7.3.1	Creating a switch statement	141
7.4	Exercise	146
8	Using Looping Statements	149
8.1	What is a loop	150
8.1.1	What is a loop	150
8.2	Create and use while loops	150
8.2.1	The while loop	150
8.2.2	Using a while loop	151
8.3	Create and use do/while loops	153
8.3.1	The do-while loop	153
8.4	Create and use for loops	154
8.4.1	Going from a while loop to a for loop	154
8.4.2	Syntax of a for loop	156
8.4.3	Parts of a for loop	158
8.5	Create and use for each loops	160
8.5.1	The enhanced for loop	160
8.5.2	Syntax of the enhanced for loop	162
8.5.3	Enhanced for loop in practice	163
8.6	Use break and continue	163
8.6.1	Terminating a loop using break	163
8.6.2	Terminating an iteration of a loop using continue	164
8.7	Nested loops	165
8.7.1	Nested loop	165
8.7.2	breaking out of and continuing with nested loops	167
8.8	Comparing loop constructs	171

8.8.1	Comparison of loop constructs	171
8.9	Exercise	172
9	Arrays and ArrayLists	173
9.1	Declare, instantiate, initialize and use a one-dimensional array	174
9.1.1	Declaring array variables	174
9.1.2	Creating and initializing array objects	175
9.2	Using arrays	177
9.2.1	Array indexing	177
9.2.2	Members of an array object	178
9.2.3	Runtime behavior of arrays	180
9.2.4	Uses of arrays	181
9.3	Create and manipulate an ArrayList	182
9.3.1	Introduction to Collections and Generics	182
9.3.2	ArrayList API	185
9.4	Traversing the elements of an ArrayList	192
9.5	ArrayList vs array	194
9.6	Exercise	195
10	Classes and Constructors	197
10.1	Relationship between an object and its members	198
10.1.1	Accessing object fields	198
10.1.2	What is "this"?	199
10.2	Apply access modifiers	202
10.2.1	Accessibility	202
10.2.2	Access modifiers	203
10.3	Apply encapsulation principles to a class	204
10.3.1	Encapsulation	204
10.4	Create and overload constructors	206
10.4.1	Creating instance initializers	206
10.4.2	Creating constructors	207
10.4.3	Overloading constructors	210
10.5	Understanding the scopes of class, instance, and local variables	213
10.5.1	Scope of variables	213
10.5.2	Scope and Visibility	213
10.5.3	Scope and Lifespan	215
10.5.4	Scopes Illustrated	216
10.5.5	Scope for the Exam	217
10.5.6	Quiz	218
10.6	Exercise	220
11	Java Methods	221
11.1	Create methods with arguments and return values	222
11.1.1	Creating a method	222
11.1.2	Returning a value from a method	223

11.1.3	Varargs	226
11.2	Create overloaded methods	228
11.2.1	Method signature	228
11.2.2	Method overloading	229
11.2.3	Method selection	230
11.3	Passing object references and primitive values into methods	234
11.3.1	Passing arguments to methods	234
11.3.2	Passing objects to methods	236
11.3.3	Returning a value from a method	239
11.4	Apply the static keyword to methods and fields	240
11.4.1	Apply the static keyword to methods and fields	240
11.4.2	Accessing static members	240
11.4.3	Accessing static members from the same class	241
11.4.4	Accessing instance members from a static method	242
11.5	Exercise	243
12	Working with the Math and Random Classes	245
12.1	Using the Math class	246
12.2	Using the Random class	247
12.3	Exercise	249
13	Working with the String Class	251
13.1	Creating and manipulating Strings	252
13.1.1	What is a "string"?	252
13.1.2	Creating Strings	252
13.1.3	String interning	257
13.1.4	Quiz	258
13.1.5	String immutability	259
13.1.6	Using the methods of the String class	259
13.1.7	Comparing strings	262
13.1.8	Quiz	264
13.2	Formatting Strings	264
13.3	Exercise	266
14	Debugging and Exception Handling	269
14.1	Create try-catch blocks and determine how exceptions alter program flow	270
14.1.1	Java exception handling	270
14.1.2	Fundamentals of the try/catch approach	271
14.1.3	Pieces of the exception handling puzzle	273
14.2	Differentiate among checked, unchecked exceptions, and Errors	278
14.2.1	Checked and Unchecked exceptions	278
14.2.2	Commonly used exception classes	281
14.3	Use try and catch blocks	282
14.3.1	Creating a method that throws an exception	282
14.3.2	Invoking a method that throws an exception	284

14.4 Exercise	286
-------------------------	-----



Introduction

I believe you have already gotten your feet wet with Java programming and are now getting serious about your goal of being a professional Java programmer. First of all, let me commend your decision to consider Java certification as a step towards achieving that goal. I can assure you that working towards acquiring **Oracle's Java Certification** will be very rewarding. Irrespective of whether you get extra credit for being certified in your job hunt or not, you will be able to speak with confidence in technical interviews and the concepts that this certification will make you learn, will improve your performance on the job. Not many people know about it but Oracle has an **entry level** Java certification named **Java Foundations Certified - Junior Associate (JFCJA)** with exam code **1Z0-811**, which is geared towards high schoolers, college goers, and Java beginners. Oracle has renamed it to **Java Certified Foundations Associate (JCFA)**. This exam hasn't been very popular because of the presence of the **Oracle Certified Associate Java Programmer** certification aka **OCAJP** with exam code **1Z0-808**, which is also meant for entry level Java professionals who want to make a career in Java programming. The OCAJP exam is also a prerequisite for getting the more advanced **Oracle Certified Professional Java Programmer (OCPJP 1Z0-809)** certification. Since the **JCFA** exam does not make one eligible to take the OCPJP exam, there was little benefit in taking this exam.

Oracle has changed the **Java certification paths** since Java 11 by eliminating the Associate level Java programming certification altogether. One just needs to pass one exam, **1Z0-819**, **1Z0-829**, or **1Z0-830**, to get the **Oracle Certified Professional Java 11**, **17**, or **21 Developer** certification. This is mostly a good thing because now one has to pay the price of just one exam to get the OCP certification. However, this has made the exam quite heavy with lots of advanced topics such as Concurrency, NIO, JDBC, Modules, Streams, and, Localization. It is very difficult for an entry level candidate to master these advanced topics. Thus, appearing for the **1Z0-819/1Z0-829/1Z0-830** exams directly is a risky gamble for entry level Java programmers.

This makes the **Oracle Certified Foundations Associate (OCFA)**, Java certification very attractive now. If you are a high schooler or a Java beginner, the **1Z0-811** exam is the best way

to prove that you have learnt the basics of Java programming. This exam costs a lot less (only \$95) than the OCPJP exam (\$245). By preparing for the OCFA certification, you will get to learn the fundamentals and you will also get a verifiable certification to show on your resume, which will help in your job hunt. You can then proceed to prepare for the more advanced OCPJP certification.

The only issue with the OCFA certification is that it is still stuck on the old Java 8 version. It is possible that Oracle may update it for a new version of Java.

I have designed this book to help one master all the topics required to pass this exam in about two months with two hours of study time every day. Of course, if you are already an experienced Java programmer, you may be over with it in a few weeks. Since the exam is currently based on Java 8, I have kept the content close to Java 8. However, if a particular topic has been affected by the new developments in Java, I have mentioned the relevant changes as well.

About the mock exams

Mock exams are an essential preparation tool for achieving a good score on the exam. Even if you know all the topics mentioned in the official exam objectives or if you have a few years of Java development experience under your belt, the exam may still trip you up if you haven't practiced answering multiple choice questions in a limited amount of time. However, having created mock exams for several certifications, I can tell you that creating good quality questions is neither easy nor quick. Even after multiple reviews and quality checks, it takes years of use by thousands of users for the questions to shed all ambiguity, errors, and mistakes. I have seen users come up with plausible interpretations of a problem statement that we could have never imagined. A bad quality mock exam will easily consume your valuable time and may also shake your confidence. For this reason, I have not created new mock exams for this book. We have a team that specializes in developing mock exams and I will recommend you to buy the exam simulator created by this team from our website <https://enthuware.com>. It is priced quite reasonably (only 9.99 USD) and has stood the test of time.

The combination of this book and Enthuware mock exams should be enough for you score above 90%.

0.1 Who is this book for?

This book is for Oracle Certified Foundations Associate Java certification exam (Exam Code 1Z0-811) takers who know a little bit about computers and programming in general. You don't need any prior Java knowledge. Some programming experience will be helpful but is not required. Many people who have not programmed earlier find out while reading their first programming book that they have an intuitive understanding of logic. They take to programming like fish to water. So, don't be afraid of jumping straight into Java programming even if you are a complete beginner. Since this book is focused on Java, I will not teach fundamentals of programming or algorithms in this book as such, but I will go through enough basics that you will be able to write simple Java programs in no time.

Before proceeding with this study guide, please answer the following questions. Remember that you don't have to be an expert in the topic to answer yes. The intention here is to check if you are at least familiar with the basic concepts. It is okay if you don't know the details, the syntax, or the typical usage.

1. Do you know what Operating System, RAM, and CPU are?
2. Do you know what a command line and File Explorer are?
3. Can you create and locate a file or a folder on your machine using File Explorer?
4. Can you execute or launch a program or a software by double clicking on the executable on your machine?
5. Can you open a text editor such as Notepad and create text files on your machine?

It would be good if know all of the above. If not, I would suggest you to quickly go through a general computer book for beginners first, and then come back to this book. I recommend **Michael Miller's "Computer Basics Absolute Beginner's Guide"** for this purpose. Alternatively, be open to google a term if you are not sure about it at any time before proceeding further while reading this book.

0.2 How is this book different from others?

Twenty years ago, when I first got into Java programming, there weren't that many books written on Java. But with Java now being the most popular programming language, there are literally thousands of books, from beginners to advanced, written on Java. However, while teaching a batch of student for the **Oracle Certified Foundations Associate (OCFA)** exam, I couldn't find one that was focused on this exam. I checked out several Java beginners books but most of them seemed way out of touch with current state of Java development. One of the beginners books spent quite a few number of pages on Applets, something that has been completely out of favor for more than fifteen years. Some spent too many pages on the history of Java, something that is interesting to know as a story but adds little value to the knowledge base of the reader. But more importantly, besides perpetuating technically imprecise clichés, most of them missed explaining fundamental aspects of the Java language and Java development. My goal in this book is to make sure you learn

the basics of Java really well and, of course, to prepare you well for the OCFA exam. Thus, this book is fundamentally different from others in the following respects:

1. **Focus on concepts** - I believe that if you get your basic concepts right, everything else falls in place nicely. While working with Java beginners, I noticed several misconceptions, misunderstandings, and bad short cuts that would affect their learning of complex topics later. I have seen so many people who manage to pass the exam but fail in technical interviews because of this reason. In this book, I explain the important stuff from different perspectives. This does increase the length of the book a bit but the increase should be well worth your time.
2. **No surgical cuts** - Some books try to stick very close to the exam objectives. So close that sometimes a topic remains nowhere close to reality and the reader is left with imprecise and, at times, incorrect knowledge. This strategy is good for answering multiple choice questions appearing on the exam but it bites the reader during technical interviews and while writing code on the job. I believe that answering multiple choice questions (MCQs) should not be your sole objective. Learning the concepts correctly is equally important. For this reason, I go beyond the scope of exam objectives as and when required. Of course, I mention it clearly while doing so.
3. **Exercises** - "Write a lot of code" is an advice that you will hear a lot. While it seems quite an easy task for experienced programmers, I have observed that beginners are often clueless about what exactly they should be writing. When they are not sure about what exactly a test program should do, they skip this important learning step altogether. In my training sessions, I give code writing exercises with clear objectives. I have done the same in this book. Instead of presenting MCQs or quizzes at the end of a topic or chapter, I ask you to write code that uses the concepts taught in that topic or chapter.

Besides, a question in the real exam generally requires knowledge of multiple topics. The following is a typical code snippet appearing in the exam:

```
int i = 10;
Long n = 20;
float f = 10.0;
String s = (String) i+n++;
```

To determine whether this code compiles or not, you need to learn four topics - wrappers, operators, String class, and casting. Thus, presenting an MCQ at the end of a topic, that focuses only on that one topic, creates a false sense of confidence. I believe it is better to focus on realistic MCQs at the end of your preparation.

4. **No wasting time** - My goal in this book is to teach you the basics of the Java language and not to teach the history of Java language or the past and the future of the Java language! I will not be wasting your time on stuff that is not important from the perspective of the exam.
5. **No Assumptions** - I do not assume that you have any prior knowledge of Java or any other programming language. I believe that you don't have to understand the programming

philosophy of other languages such as C/C++ before learning Java just because Java came a long time after them. Comparisons with other languages are helpful to those who already know other programming language(s) but it is less confusing for a new programmer to learn Java without worrying about the features of other languages.

0.3 How is this book organized?

This book consists of seventeen chapters plus this introduction at the beginning. Other than the first chapter "Kickstarter for the Beginners", the chapters correspond directly to the official exam objectives. The sections of a chapter also correspond directly to the items of exam objectives in most cases. Each chapter lists the exam objectives covered in that chapter at the beginning and includes a set of coding exercises at the end. It would be best to read the book sequentially because each chapter incrementally builds on the concepts discussed in the previous chapters. I have included simple coding exercises throughout the book. Try to do them. You will learn and remember the concept better when you actually type the code instead of just reading it. If you have already had a few years of Java development experience, you may go through the chapters in any order.

Conventions used in this book

This book uses certain typographic styles in order to help you quickly identify important information. These styles are as follows:

Code font - This font is used to differentiate between regular text and Java code appearing within regular text. Code snippets containing multiple lines of code are formatted as Java code within rectangular blocks.

Red code font - This font is used to show code that doesn't compile. It could be because of incorrect syntax or some other error.

Output code font - This font is used to show the output generated by a piece of code on the command line.

Bold font - I have highlighted important words, terms, and phrases using bold font to help you mentally bookmark them. If you are cruising through the book, the words in bold will keep you oriented besides making sure you are not missing anything important.

Please note that colors are visible only in the eBook/PDF/Kindle version. The paperback version is printed in black and white with colors displayed as shades of gray.

Note -

Things that are not completely related to the topic at hand are explained in notes. I have used notes to provide additional information that you may find useful on the job or for technical interviews but will most likely not be required for the exam.

Exam Tip:

Exam Tip

Exam Tips contain points that you should pay special attention to during the exam. I have also used them to make you aware of the tricks and traps that you will encounter in the exam.

Asking for clarification

If you need any clarification, have any doubt about any topic, or want to report an error, feel free to ask on our dedicated forum for this book - <http://enthuware.com/forum>. If you are reading this book on an electronic device, you will see this icon  beside every topic title. Clicking on this icon will take you to an existing discussion on that particular topic in the same forum. If the existing discussion addresses your question, great! You will have saved time and effort. If it doesn't, post your question with the topic title in the subject line. We use the same mechanism for addressing concerns about our mock exam questions and have received tremendous appreciation from the users about this feature.

0.4 General tips for the exam

Here is a list of things that you should keep in mind while preparing for the exam:

1. **Number of correct options** - Every question in the exam will tell you exactly how many options you have to select to answer that question correctly. Remember that there is **no negative marking**. In other words, marks will not be deducted for answering a question incorrectly. Therefore, do not leave a question unanswered. If you don't know the answer, select the required number of options anyway. There is a slight chance that you will have picked the correct answer. Some people believe that picking the same option, for example option 2, for all such questions is a good idea.

There is **no credit for partial answer** either. You must select all correct options to get the credit for answering that question correctly.

2. **Eliminate wrong options** - Even better than not leaving a question unanswered is to make intelligent guesses by eliminating obviously incorrect options. You may see options that are contradictory to each other. This makes it a bit easy to narrow down the correct options.
3. **Mark and Move** - You have to answer a fixed number of questions (60) in a fixed amount of time (120 minutes). That means, you have about two minutes to answer each question. One approach could be that you move to the next question after every 2 minutes. However, this is

not a very good approach. The exam has easy as well as tough questions. Some topics (such as loops) are, by nature, very hard and require a lot more time to answer. Furthermore, you never know which type of question will you get at the beginning of the exam. If you get a few time consuming questions at the start, you will start feeling the time pressure and that will affect your ability to answer on the subsequent questions. It is very easy to answer a simple question incorrectly when you are stressed. A better approach is to take a quick look at the question and judge whether you can answer it quickly with confidence. If not, just mark the question (there is a checkbox for marking a question on the top right corner) and move to the next question. This way, you will be able to answer all easy questions without pressure. The goal here is to make sure that you don't trip on the easy questions. Once you are done with the last question. Go to the Review screen and start solving the "marked" questions. You will now have enough time to solve the tough and time consuming questions because you never took full 2 minutes on the easy questions. Even if you are not able to answer a few questions at the end, you know that they were tough questions and there was a high chance of answering them incorrectly anyway.

4. **Read all options** - Sometimes, one feels so, confident about an option that they select it believing it to be correct and move on to the next question. However, the questions and the options in this certification exam can sometimes be tricky. The options are designed to confuse you and therefore, it is absolutely critical to read all options even if you feel you have found the right option. It is better to take a moment to rule out the rest of the options.
5. **Must/Cannot/Never/Always** - You should be very careful about selecting an option that has such strong words because there is almost always an exception to a rule. If you see two options where one says you "cannot" do X, while the other says, you "may" do X, and if you have no idea about the topic, select the second option because there is a good chance that a workaround to do X exists.
6. **Do not fight the question writer** - Questions are developed by humans and it is possible that you may find a question to be ambiguous or incorrect. You may also feel that more than one options are correct. In such situation, pick the option that you feel is the best. Do not go with the option that you feel is correct in an obscure situation. Go with the most common scenario.
7. **Ignore minor/trivial errors** - Although all information given in the question is important, you may encounter a question with an obvious typo or mistake. It is best to ignore such errors because the exam doesn't test you on such typos. The exam is tough but not unfair. For example, if you see `System.out.println` misspelled as `Sytem.out.println`, ignore that mistake.
8. **Code Formatting** - You may not find nicely formatted code in the exam. For example, you may expect a piece of code nicely formatted like this:

```
if(flag){
    while(b<10){
    }
}else if(a>10) {
    invokeM(a);
}
```

```
}  
else{  
    System.out.println(10);  
}
```

But you may get the same code formatted like this:

```
if(flag){  
    while(b<10){ }  
    } else  
if(a>10) { invokeM(a); }  
else {    System.out.println(10); }
```

They do this most likely to save space. But it may also happen inadvertently due to variations in display screen size and resolution.

9. **Assumptions** - Several questions give you partial code listings, aka "code snippets". For example, what will the following code print?

```
ArrayList al = new ArrayList();  
al.remove(0);  
System.out.println(al);
```

Obviously, the code will not compile as given because it is just a code fragment. You have to assume that this code appears in a valid context such as within a method of a class. You also need to assume that appropriate import statements are in place.

Do not fret over the missing stuff. Assume that all such things such as the source code file name, the directory, classpath/path, are valid. Just focus on the code that is given and assume that everything else is irrelevant and is not required to arrive at the answer.

10. **Tricky Code** - You will see really weird looking code in the exam. Code that you may never even see in real life. You will feel as if the exam is about puzzles rather than Java programming. To some extent, that is correct. If you have decided to go through the certification, there is no point in questioning the relevance. If you feel frustrated, I understand. Please feel free to vent out your anger on our forum and get back to your studies!

That's about it. I hope this book helps you become a better Java programmer besides getting you the certification.

0.5 Official Exam Details and Exam Objectives

The following are the official exam details published by Oracle as of 1st July 2019. As mentioned before, Oracle may change these details at any time. They have done it in the past. Several times. Therefore, it would be a good idea to check the official exam page at https://education.oracle.com/java-foundations/pexam_1Z0-811 during your preparation.

Exam Details

Duration: 120 Minutes

Number of Questions: 60

Passing Score: 65%

Format: Multiple Choice

Exam Price: USD 95 (varies by country)

Exam Objectives

What Is Java?

1. Describe the features of Java
2. Describe the real-world applications of Java

Basic Java Elements

1. Identify the conventions to be followed in a Java program
2. Use Java reserved words
3. Use single-line and multi-line comments in Java programs
4. Import other Java packages to make them accessible in your code
5. Describe the java.lang package

Working with Java Operators

1. Use basic arithmetic operators to manipulate data including +, -, *, /, and %
2. Use the increment and decrement operators
3. Use relational operators including ==, !=, > , > =, <, and <=
4. Use arithmetic assignment operators
5. Use conditional operators including &&, ||, and ?
6. Describe the operator precedence and use of parenthesis

Working with the Random and Math Classes

1. Use the Random class
2. Use the Math class

Using Looping Statements

1. Describe looping statements
2. Use a for loop including an enhanced for loop

3. Use a while loop
4. Use a do- while loop
5. Compare and contrast the for, while, and do-while loops
6. Develop code that uses break and continue statements

Arrays and ArrayLists

1. Use a one-dimensional array
2. Create and manipulate an ArrayList
3. Traverse the elements of an ArrayList by using iterators and loops including the enhanced for loop
4. Compare an array and an ArrayList

Java Methods

1. Describe and create a method
2. Create and use accessor and mutator methods
3. Create overloaded methods
4. Describe a static method and demonstrate its use within a program

Java Basics

1. Describe the Java Development Kit (JDK) and the Java Runtime Environment (JRE)
2. Describe the components of object-oriented programming
3. Describe the components of a basic Java program
4. Compile and execute a Java program

Working with Java Data Types

1. Declare and initialize variables including a variable using final
2. Cast a value from one data type to another including automatic and manual promotion
3. Declare and initialize a String variable

Working with the String Class

1. Develop code that uses methods from the String class
2. Format Strings using escape sequences including %d, %n, and %s

Using Decision Statements

1. Use the decision making statement (if-then and if-then-else)
2. Use the switch statement
3. Compare how `==` differs between primitives and objects
4. Compare two String objects by using the `compareTo` and `equals` methods

Debugging and Exception Handling

1. Identify syntax and logic errors
2. Use exception handling
3. Handle common exceptions thrown
4. Use try and catch blocks

Classes and Constructors

1. Create a new class including a main method
2. Use the private modifier
3. Describe the relationship between an object and its members
4. Describe the difference between a class variable, an instance variable, and a local variable
5. Develop code that creates an object's default constructor and modifies the object's fields
6. Use constructors with and without parameters
7. Develop code that overloads constructors

0.6 Feedback, Errata, and Reviews

If you have any query regarding the contents of this book or if you find any error, please do let me know on <https://enthuware.com/forum>.

All confirmed errors are listed here: <https://enthuware.com/811/errata.php>.

Since this book is published on "print on demand" basis, an updated version, with all enhancements and fixes, is published every few weeks. You may check whether you have the latest version by comparing build number mentioned title page of your copy shown in book description on <https://enthuware.com/811/amazon.php>. If you are using the Kindle ebook version and have an older version, you can get the most recent version by requesting Amazon support. They will send the updated version on your device upon your request.

I hope you enjoy reading this book. If you learn a few things and find it interesting, I would be very grateful if you would consider leaving a review on <https://enthuware.com/811/amazon.php> with a few kind words. If you have received a review copy of this book, please mention so, in your review.

thank you,
Hanumant Deshmukh

